

# Lesson 6: Algorithmic Thinking With Everyday Tasks

Generated Jul 7, 2026 1:26 PM

---

## What We'll Learn

---

By the end of this lesson, we can:

- Define algorithm using daily-life examples.
- Arrange steps in correct order.
- Explain why order matters.

## Words We'll Use

---

- Algorithm - ordered steps to solve a problem.
- Step - one action in a process.
- Sequence - correct order.
- Input - information needed.
- Output - result after steps.

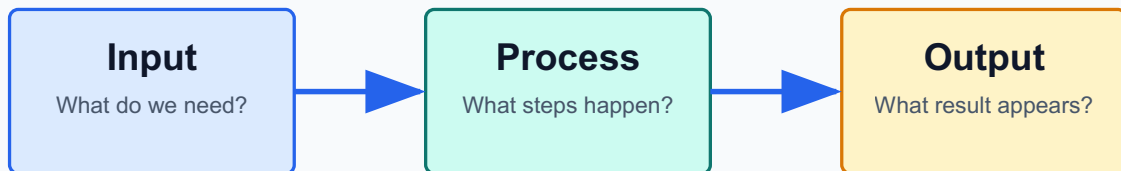
## Before We Start

---

Describe how you log in to a computer. What happens if one step is skipped?

# Let's Understand

## Algorithm Planning: IPO



Example: numbers 5 and 7 -> add them -> display 12

Let's work through Lesson 6: Algorithmic Thinking With Everyday Tasks together. Our focus is algorithms as ordered steps for completing familiar tasks. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- An algorithm is an ordered list of steps that solves a problem or completes a task.
- Before we code, we identify the input, process, and output so our solution has direction.
- Good steps are clear enough that another person can follow them without guessing.
- The order matters because computers follow instructions exactly as written.
- We can test an algorithm with sample values before writing Java code.
- A strong algorithm is specific, complete, logical, and easy to revise.

How we study it step by step:

- First, we connect the lesson to a familiar situation: Describe how you log in to a computer. What happens if one step is skipped?
- Next, we name the important parts so everyone uses the same vocabulary.
- Then, we study Daily algorithms and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Order and sequence and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Input and output and ask: what does it do, where do we see it, and how can we check it?
- After that, we compare a correct example with a common wrong version.
- We trace the example slowly before running, drawing, or submitting anything.
- Finally, we explain the result in our own words so the activity is not just copying.

Rules and patterns we should remember:

- Rule: Use action words such as read, compute, compare, display, repeat, or stop. Example: Step 1: Read firstNumber. Step 2: Read secondNumber. Step 3: Compute sum. Check: Which word tells us the action in Step 3?
- Rule: Put one clear action per step when possible. Example: Better: Read grade, then Compare grade to 75; unclear: Check everything. Check: Which version can a beginner follow?
- Rule: Include the needed input before the process step. Example: For sum of two numbers, we first need number1 and number2 before computing sum. Check: What information must we ask for first?
- Rule: Include the output after the process or decision step. Example: After computing  $\text{sum} = 5 + 7$ , we display 12. Check: What should the user see at the end?
- Rule: Test the algorithm with at least one sample value. Example: If number1 is 5 and number2 is 7, the expected sum is 12. Check: Does our step list produce 12?

Common mistakes we will watch for:

- Writing a paragraph instead of ordered steps.
- Starting with computation before identifying inputs.
- Skipping the output step.
- Combining several actions into one vague step.
- Not testing whether another person can follow the algorithm.

Mini-check before practice:

- What is our input?
- What process happens to the input?
- What output should appear?
- Are the steps clear enough for a classmate to follow?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

# Let's Practice

---

Let's practice with these activities:

Activity 1: Everyday Steps To Algorithm Type: individual Time: 8 minutes Instruction: Turn a familiar task into ordered steps. Student task:

- Choose logging in to a computer or submitting an assignment.
- Write 8-10 numbered steps.
- Mark the input/materials and expected output.

Output: Submit an ordered algorithm with input/materials and output.

Activity 2: Arrange The Algorithm Type: pair Time: 8 minutes Instruction: Put scrambled steps in the correct order. Student task:

- Arrange: display welcome screen, type password, click login, open computer, type username.
- Rewrite the corrected order.
- Explain why one step must come first.

Output: Submit corrected order and one explanation.

Activity 3: Missing Step Detective Type: pair Time: 8 minutes Instruction: Find missing steps that make an algorithm unclear. Student task:

- Read: Open IDE, run program, submit output.
- Add at least three missing steps.
- Check if a beginner could follow it.

Output: Submit improved algorithm.

Activity 4: Partner Test Type: pair Time: 10 minutes Instruction: Test whether another person can follow your steps exactly. Student task:

- Exchange algorithms.
- Follow your partner's steps without adding hidden steps.
- Mark confusing steps.
- Revise your own algorithm.

Output: Submit original steps, partner feedback, and revised steps.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check algorithm vocabulary. Student task:

- Define algorithm in your own words.
- Write one reason order matters.
- Write one example of output.

Output: Submit three short answers.

## Now We Apply

---

Now we apply it through these tasks:

1. Write an algorithm for submitting an online assignment.
2. Create an algorithm for preparing your Java workspace.
3. Revise your algorithm after partner feedback.

## Challenge Task

---

Create a clear 10-step algorithm for a daily school or computer task with title, input/materials, ordered steps, expected output, and one safety note.

## How Our Work Will Be Checked

---

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

## Let's Reflect

---

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

## How We Show Learning

---

Evidence we will use to check learning:

- Steps are ordered and specific.
- Algorithm can be followed by another person.
- Output/result is clear.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

## Student Activities

---

Everyday Steps to Algorithm. Arrange the Algorithm. Missing Step Detective. Too Broad or Clear. Partner Test. Reflection Check. Write an algorithm for submitting an online assignment. Create an algorithm for preparing your Java workspace. Revise your algorithm after partner feedback.

## Resources / References

---

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.